

2-Day Trial: Fermi Podcast Companion

Candidate Brief | Two-Day Work Trial

1) Overview

Fermi Podcast is a catalogue of long-form physics podcast series. Its episodes explore subjects such as condensed matter, quantum computing, astrophysics, and other frontiers of physics.

For this trial, Fermi will provide a selected collection of raw audio from Fermi Podcast. Build a conversational AI product that allows a learner to interact with and learn from this collection.

You will receive the audio files only. You will not receive transcripts, processed data, a product specification, or a prescribed technical approach. Deciding what to build and breaking the problem into a workable system are central parts of the trial.

2) What We Are Evaluating

- Can you turn an open-ended product problem into a clear and valuable user experience?
- Can you use AI tools and web research to learn unfamiliar techniques and make them work together?
- Can you make and explain sensible technical choices under a tight time limit?
- Can you build a complete working system within two days rather than a collection of disconnected experiments?
- Can you determine whether the system actually works, find its failures, and improve it using evidence?

Prior experience with this exact problem is not required. We are evaluating how effectively you learn and build with the tools available today.

3) Input and Freedom

You will receive:

- 3 raw Fermi Podcast audio files, totalling no more than 3 hours.
- A fixed Fermi-provided budget for AI API usage.

You may use third-party AI APIs, open-source libraries, AI coding tools, and web research. You may choose any models, services, programming languages, frameworks, storage, or interface that you believe fit the problem.

You must start from the supplied raw audio. You may not use YouTube's or another hosting platform's existing closed captions, transcripts, or caption/transcript APIs as input to your system.

You may not upload the files to an existing end-user product and submit that product or screenshots as your solution. The submission must contain the system you built.

4) Minimum Product Outcome

The submission must be a runnable conversational experience built from the supplied raw audio. At minimum:

- A learner can have a useful natural-language conversation across the supplied episodes.
- The product's responses remain faithful to the supplied audio and can be checked against it.

- The experience does more than display a transcript or a one-time summary.

Beyond this minimum, you must decide who the product is for, which user problems matter most, what the experience should do, and how it should work.

Examples of the kinds of learner interactions the product could support:

- "Which episodes should I listen to if I want to understand superconductivity, and why?"
- "What does 'more is different' mean in the first Condensed Matter episode? Explain it simply."
- "Compare what these episodes say about noise in quantum systems."
- "I did not understand the example you just gave. Walk me through it step by step."
- "Take me to the part of the original audio that supports this answer."
- "Do these episodes cover this topic?" If the supplied collection does not, the product should not invent an answer.

These are examples, not a feature checklist. You may identify a different learner need and design a better experience around it, provided the minimum product outcome is met.

5) Deliverables

- Runnable source code.
- A short README with setup and one-command run instructions.
- A short product note defining the intended user, the problem you chose to solve, and why you chose it.
- The artifacts required to run the system, or a reproducible command that creates them from the supplied audio.
- A runnable evaluation system, including its evaluation cases and a one-command way to execute it against your product.
- A brief EVAL.md explaining the evaluation system, success criteria, results, raw examples, failure analysis, and measured improvement.
- A short demo of the working product.

6) Evaluation Requirement

You must build an evaluation system for the product. Evaluation is not an optional write-up after implementation, and EVAL.md alone is not the evaluation system.

You must:

1. Define what success means for the experience you chose to build.
2. Create a varied evaluation set of at least 10 cases beyond your demo examples.
3. Implement a repeatable runner that executes those cases against the product and preserves the raw inputs and outputs.
4. Run your initial system on that set and preserve the raw results.
5. Manually inspect concrete successes and failures.
6. Make one meaningful improvement based on the observed failures.
7. Re-run the same evaluation and explain what improved, regressed, or remained unresolved.

Fermi will also evaluate the submitted system on unseen interactions using the supplied podcast collection. We will inspect the actual outputs and their relationship to the source audio, not only the metrics reported in your README.

7) Grading Rubric (100 Points)

Category	What we are looking for	Weight
Problem Framing and Product Judgment	A valuable learner problem, clear intended user, sensible scope, and coherent product experience	20
AI Engineering and System Design	Effective use of models and tools, sound technical decisions, complete end-to-end execution, and justified trade-offs	25
Conversation Quality and Trustworthiness	A useful learner experience, faithful use of the supplied audio, source verifiability, and responsible handling of unsupported requests	20
Evaluation System and Improvement	A runnable evaluation system, meaningful success criteria, varied cases, raw evidence, honest failure analysis, and a measured baseline-to-improvement comparison	25
Code Quality and Runability	Reproducible setup, working commands, understandable code, and clear communication	10

A polished interface cannot compensate for a system that is not grounded in the supplied audio. A reported aggregate score cannot compensate for unsupported outputs visible in the raw examples.

8) Non-Goals

- Production deployment, authentication, billing, or multi-user infrastructure.
- Supporting a large or continuously changing podcast catalogue.
- Training a speech or language model from scratch.
- A highly polished frontend.
- Audio editing, podcast generation, or voice cloning.

9) Timebox

Plan for **2 days**. We do not expect a production system. We expect a thoughtful product decision, a working end-to-end implementation, evidence showing where it succeeds and fails, and one justified improvement.